

Perceptron

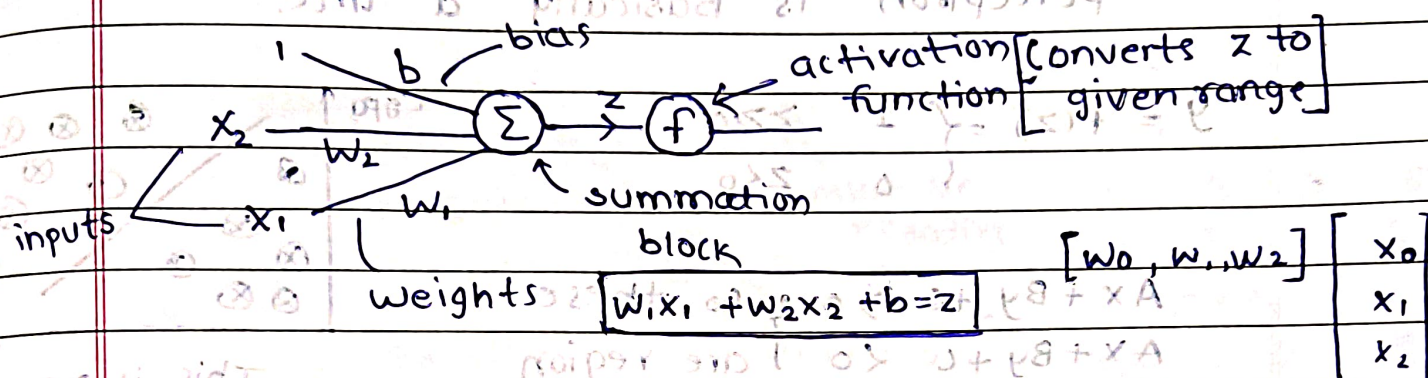
Types of Neural Network.

Date _____

Page _____

- It is basically an algorithm used for supervised machine learning.
- We can also call it a mathematical model / function

Design of Perceptron



weights

- weights defines the feature importance
- It tells us how strong the connection is.
- large weights - strong connection
- lower weights - weaker connection

the model relies more on that specific connection to make its prediction	tells indicates that the input feature has very little influence on the activation function of the connected neuron.
--	--

zero weights - the connection is effectively dead

Near zero (0.01 to 0.1) - A weak connection

Near one (0.9 to 1.0) - A strong positive connection

Negative weights: An inhibitory connection

Geometric Intuition

Date _____

Page _____

so, Perceptron is:

→ binary classifier

→ by creating 2 regions

$$w_1 \Rightarrow A$$

$$w_2 \Rightarrow B$$

$$b \Rightarrow C$$

$$x_1 \Rightarrow x$$

$$x_2 \Rightarrow y$$

$$Ax + by + c = 0 \rightarrow \text{Eqn of Line}$$

so,

perceptron is basically a line.

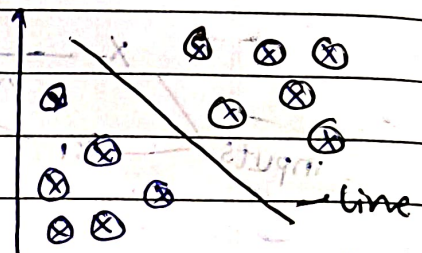
$$y = f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

$$Ax + By + c > 0$$

$$Ax + By + c < 0$$

so, these

are region



This is 2D

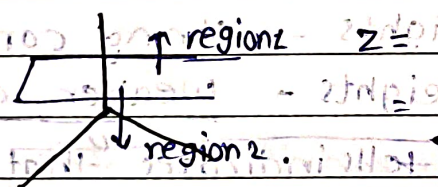
we can say,

Perceptron creates region & divides the classes

to the regions

For

3D →



$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

$$Ax + By + Cz + b \geq 0$$

2D →

line

3D →

plane

4D →

hyperplane

so,

Limitation of Perceptron:

Perceptron can be use classify only linear/

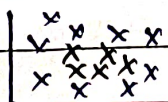
sort of linear data.

જેવા કે perceptron અશક data નો classify કરે
શકે તે જાણી તે line ને sepearate કરે શકે.

If we have completely non linear data,

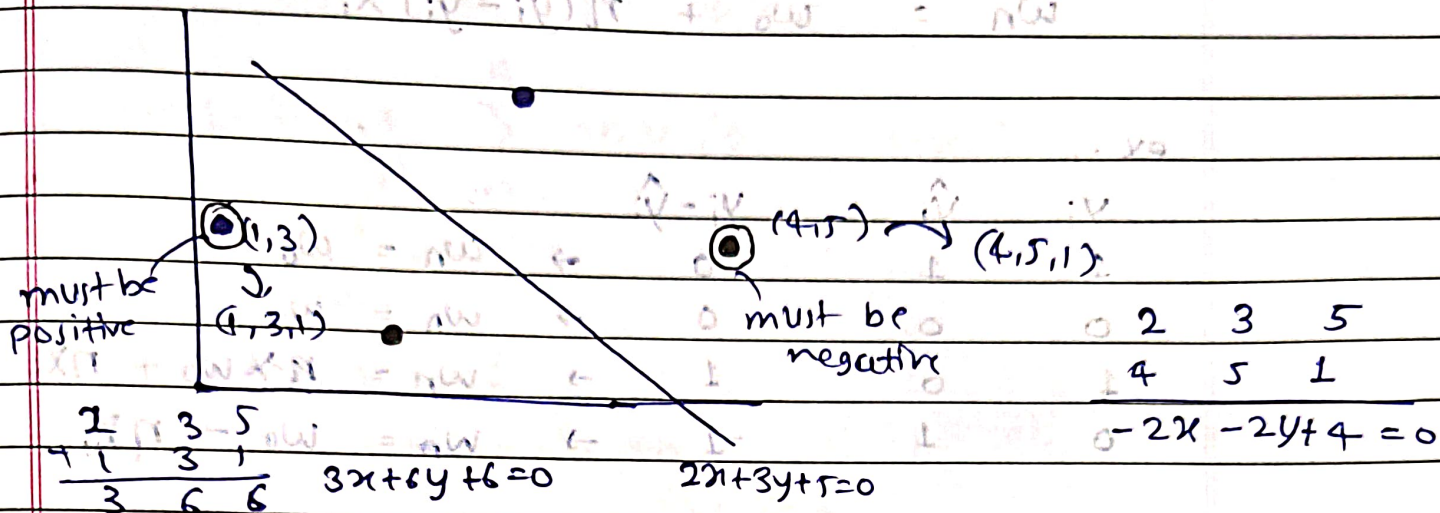
then perceptron fails.

like



→ can not
separate
using line.

- If a negative point is in positive region we will subtract it's coordinates from the line's coefficients.



- If positive point is in negative region, we will add it's coordinates to the coefficients of line

Algorithm to change

epoch = 1000, $\eta = 0.01$

for i in range(epoch):

randomly select a student

if $x_i \in N$ and $\sum_{i=0}^n w_i x_i \geq 0$

$$w_{\text{new}} = w_{\text{old}} - \eta x_i$$

if $x_i \in P$ and $\sum_{i=0}^n w_i x_i < 0$

$$w_{\text{new}} = w_{\text{old}} + \eta x_i$$

$x_i \in N \rightarrow x_i$ belongs to Negative region

$x_i \in P \rightarrow x_i$ belongs to positive region

$\sum w_i x_i \geq 0 \rightarrow$ model predicted as positive

$\sum w_i x_i < 0 \rightarrow$ model predicted as negative

simplified algorithm:

for i in range(epochs)

select a random student (i).

$$w_n = w_0 + \eta(y_i - \hat{y}_i) X_i$$

ex.,

y_i	$\hat{y}_i$	$y_i - \hat{y}_i$	
1	1	0	$\rightarrow w_n = w_0$

0	0	0	$\rightarrow w_n = w_0$
---	---	---	-------------------------

1	0	1	$\rightarrow w_n = \eta X_i w_0 + \eta X_i$
---	---	---	---

0	1	-1	$\rightarrow w_n = w_0 - \eta X_i$
---	---	----	------------------------------------

The perceptron trick has two main limitations:

- **Lack of Quantifiable performance:** The trick does not allow you to measure or compare how well your model is performing. Because it relies on simply moving the decision boundary based on misclassified points, you cannot tell if the resulting line is best possible solution or if a better one exists.
- **Convergence Issues:** Because the training process involves randomly selecting data points, there is no guarantee that the model will reliably converge on a stable or optimal solution every time you run it. In certain scenarios, the line might fail to settle correctly.